
SentinelShark

Network Intrusion Detection & Analysis System

Version 2.4.1 · MIT License · Cross-Platform Desktop

Comprehensive Technical Documentation

Project Synopsis · System Architecture · UI Design · Engineering Portfolio

Author & Lead Developer: Debdoot Manna

Target Operating Systems: macOS 12+ (Darwin) and Windows 10/11 (NT)

Core Codebase Technologies: Python 3.10+, C# .NET 8 (WPF), TypeScript

Repository Location: <https://github.com/DebdootManna/sentinelshark>

Documentation Date: August 31, 2026

Core Frameworks: PyQt6 · qasync · PyShark · httpx · cachetools · psutil · .NET 8 WPF

Threat Intelligence Feeds: AbuseIPDB · VirusTotal · IPinfo · Shodan & InternetDB

Contents

1	Project Synopsis	5
1.1	Executive Summary	5
1.2	The Problem: Limitations of Legacy Network Analyzers	5
1.3	Proposed Solution & Core Innovations	5
1.4	High-Level Summary Metrics	6
2	Project Definition & Requirements	7
2.1	Project Mission & High-Level Objectives	7
2.2	System Scope	7
2.2.1	In Scope Capabilities	7
2.2.2	Out of Scope Boundaries	8
2.3	Stakeholder Matrix	8
2.4	Detailed Functional Requirements	8
2.5	Non-Functional Requirements & Performance Targets	9
3	System Architecture & Data Engineering	11
3.1	Three-Tier Layered Architecture Pattern	11
3.2	Asynchronous Concurrency Model: qasync & QThread	11
3.3	Technology Stack Specifications	11
3.4	Python Module Organization	11
3.5	Packet Capture Lifecycle & Multi-Tier Fallback	11
3.6	Threat Intelligence Pipeline Architecture	13
3.7	Unified Packet Data Model	13
4	Core Features & Threat Intelligence	17
4.1	Quad-Provider Threat Intelligence Architecture	17
4.2	Shodan Hybrid Fallback Mechanism	17
4.3	Composite Threat Scoring & Color Triage	17
4.4	BPF Filter Sanitization	18
4.5	Supported Network Protocols	18
5	User Interface & Visual Design	20
5.1	UI Design Philosophy & Layout Architecture	20
5.2	Control Toolbar & Interactive Controls	21
5.3	Live Packet Table (<code>PacketTable</code>)	21
5.4	Live Telemetry & Statistics Dashboard (<code>StatsPanel</code>)	21
5.5	Protocol Dissection Tree (<code>PacketDetailView</code>)	21
5.6	Hexadecimal & ASCII Byte Inspector (<code>HexView</code>)	22
5.7	Real-Time Bandwidth Sparkline (<code>SparklineWidget</code>)	22
5.8	Network Interface Selection Dialog	22
5.9	API Key Configuration Dialog	22
5.10	UI Component Summary	22
5.11	Design Color Palette & Theme Tokens	22
6	Configuration & Secret Management	24
6.1	Two-File Configuration Architecture	24

6.2	Automatic Legacy Configuration Migration	24
6.3	Configuration Parameter Reference	24
6.4	TShark Auto-Discovery Hierarchy	25
7	Testing & Quality Assurance	26
7.1	Testing Philosophy & Verification Methodology	26
7.2	Detailed Unit Test Case Specifications	26
7.3	Executing the Test Suite	26
8	Project Portfolio & Engineering Analysis	28
8.1	Project Genesis & Engineering Evolution	28
8.2	Phased Development Roadmap & Gantt Timeline	28
8.3	Software Architecture Metrics & Code Distribution	28
8.4	Cross-Platform Engineering Analysis	30
8.5	Performance Engineering & Benchmark Analysis	30
8.6	Security Posture & Credential Isolation	30
8.7	Comprehensive Deployment & Runbook	31
8.8	Future Roadmap & Enterprise Extensions	31
9	.NET / WPF Companion Application	32
9.1	Architecture Overview	32
	References	34

List of Figures

3.1	SentinelShark Layered System Architecture	11
3.2	Packet Capture Engine Execution Lifecycle	12
3.3	Threat Intelligence Enrichment Pipeline	15
4.1	Shodan Hybrid Fallback Execution Flow	18
5.1	SentinelShark Desktop Application User Interface	20
8.1	SentinelShark Engineering Development Timeline	28
8.2	Code Size Distribution of Primary Python Modules	29

List of Tables

1.1	SentinelShark Key Performance and System Metrics	6
2.1	Project Objectives and Completion Status	7
2.2	System Stakeholder Roles and Value Propositions	8
2.3	Comprehensive Functional Requirements (FR-01 through FR-20)	8
2.4	Non-Functional Requirements and Operational Constraints	10
3.1	Complete Technology Stack Specifications	13
3.2	Python Application Modules and Line Counts	14
3.3	Unified Packet Dictionary Structure	16
4.1	Threat Intelligence API Provider Specifications	17
4.2	Threat Score Classification and Visual Color Coding	18
4.3	BPF Shortcut Translation Engine	19
4.4	Protocol Decoding and Dashboard Color Scheme	19
5.1	PyQt6 UI Component Reference	23
5.2	Official SentinelShark Theme Color Palette	23
6.1	Complete Configuration Options	24
7.1	Automated Unit Test Case Specifications	26
8.1	Comprehensive Codebase Size Breakdown	29
8.2	Operating System Compatibility Matrix	30
9.1	SentinelSharkDotNet Build Properties	32
9.2	.NET Application Module Organization	33

Chapter 1

Project Synopsis

1.1 Executive Summary

SentinelShark is a high-performance, modern desktop **Network Intrusion Detection and Analysis System (NIDS)**, engineered as an intelligent, real-time threat-correlated alternative to classic packet analyzers such as Wireshark. While conventional sniffers provide raw packet inspection, they leave security analysts with the cumbersome manual burden of extracting IP addresses and looking them up across disparate reputation feeds. SentinelShark bridges this gap by unifying live packet capture, deep protocol parsing (L2 through L7), and automated asynchronous threat intelligence enrichment from four premier security providers: **AbuseIPDB**, **VirusTotal**, **IPinfo**, and **Shodan**.

Designed for security analysts, incident responders, network administrators, and cybersecurity researchers, SentinelShark transforms raw network telemetry into actionable security insights in real time. The software automatically calculates composite threat scores, color-codes suspicious flows, extracts payload hashes (MD5 and SHA256), decodes application protocols, and presents network health through an ergonomic dark-mode user interface.

To ensure wide enterprise applicability, SentinelShark includes both a primary cross-platform Python 3.10+ / PyQt6 application and a standalone native **C# .NET 8 / WPF** companion application for Windows environments.

1.2 The Problem: Limitations of Legacy Network Analyzers

Network environments generate vast amounts of high-velocity traffic. When diagnosing potential compromises, data exfiltration, or denial-of-service attempts, security personnel face critical bottlenecks:

- **Absence of Automated Threat Context:** Standard packet analyzers capture raw frames but offer zero correlation with global threat databases. Analysts must copy each public IP address manually into browser tabs to verify reputation.
- **Inefficient Incident Response Triage:** Under high packet volumes, distinguishing benign cloud services from malicious Command-and-Control (C2) servers or Tor exit nodes is challenging without visual risk triage.
- **Binary Dependency Fragility:** Traditional packet sniffers crash or become completely unusable when low-level capture drivers (like Npcap or libpcap) are absent or when users lack administrative privilege.
- **API Rate Limit Exhaustion:** Ad-hoc querying of external threat APIs leads to quota burnout without intelligent rate limiting, local TTL caching, and exponential backoff handling.

1.3 Proposed Solution & Core Innovations

SentinelShark resolves these operational challenges through three architectural pillars:

1. **Resilient Zero-Friction Capture Architecture:** The engine searches for installed TShark binaries, seamlessly degrades to PyShark, and includes an internal high-fidelity

Mock Traffic Generator (30–70 packets/sec). This ensures that researchers can test and demonstrate detection workflows without special hardware or elevated privileges.

2. **Asynchronous Non-Blocking Threat Intelligence Pipeline:** External threat lookups run on non-blocking worker threads managed by `qasync` and `httpx`. An in-memory `TTLCache` with LRU eviction prevents duplicate requests, and an exponential backoff engine protects API quotas against HTTP 429 limits.
3. **Ergonomic Visual Triage Interface:** Built with a Figma-inspired dark cyber design, SentinelShark visually categorizes every flow using color-coded severity tiers (Safe Green, Warning Amber, Critical Crimson), real-time throughput sparklines, collapsible telemetry cards, and multi-layer protocol decoders.

1.4 High-Level Summary Metrics

Table 1.1: SentinelShark Key Performance and System Metrics

System Metric	Measurement	Engineering Detail / Implementation
Release Version	v2.4.1	Production-ready desktop release
Primary Platform	Python 3.10+ / PyQt6	Cross-platform macOS (Darwin) and Windows 10/11
Companion Engine	C# / .NET 8 WPF	Single-file self-contained native executable for Windows
Live Sniffing Throughput	>500 packets/sec	Line-buffered direct TShark JSON streaming subprocess
Mock Simulation Speed	30–70 packets/sec	High-fidelity simulated multi-protocol generator
Threat Intelligence APIs	4 External Services	AbuseIPDB v2, VirusTotal v3, IPinfo JSON, Shodan Host/InternetDB
Supported Protocols	10+ Decoders	DNS, HTTP, HTTPS/TLS, SSH, TCP, UDP, ICMP, ARP, QUIC
Threat Cache TTL	24 Hours	In-memory TTLCache (2,000-IP capacity with LRU eviction)
Default API Rate Limit	30 requests/min	Configurable token bucket with exponential HTTP 429 backoff
Unit Test Suite	13 Automated Tests	Zero-dependency Python <code>unittest</code> verification suite
Total Codebase Size	~9,145 LoC	Python (4.8k) + .NET (2.8k) + React UI (1.1k) + Tests (309)

Chapter 2

Project Definition & Requirements

2.1 Project Mission & High-Level Objectives

The primary mission of SentinelShark is to empower security analysts with instant, automated visibility into network traffic risk without introducing operational complexity or high infrastructure overhead.

Table 2.1: Project Objectives and Completion Status

ID	Priority	Objective Description	Status
O-01	Critical	High-throughput packet capture via direct TShark JSON streaming	Complete
O-02	Critical	Protocol dissection across OSI layers L2 through L7 into structured models	Complete
O-03	Critical	Asynchronous threat intelligence query pipeline across 4 external APIs	Complete
O-04	Critical	Robust fallback mechanism to simulated traffic (Mock Mode) when TShark absent	Complete
O-05	High	Standard Wireshark-compatible binary PCAP/PCAPNG file import and export	Complete
O-06	High	Real-time BPF capture filter sanitization and live table query filtering	Complete
O-07	High	Visual threat triage via dynamic score calculation and row color coding	Complete
O-08	High	In-memory TTL caching with completeness validation to conserve API quotas	Complete
O-09	Medium	Full cross-platform compatibility on macOS 12+ and Windows 10/11	Complete
O-10	Low	Deliver a native C# .NET 8 / WPF companion application for Windows	Complete

2.2 System Scope

2.2.1 In Scope Capabilities

- Live host network interface sniffing using line-buffered TShark JSON subprocess streams.
- Standard binary PCAP and PCAPNG file import with prompt-driven automated threat analysis.
- Native binary PCAP export using low-level `struct.pack` libpcap file formatting.
- Comprehensive protocol dissection: Ethernet, IPv4, IPv6, TCP, UDP, DNS, HTTP, HTTPS/TLS, SSH, ICMP, ARP, and QUIC.
- Automated threat enrichment querying AbuseIPDB, VirusTotal, IPinfo, and Shodan.
- Smart private IP filtering bypassing non-routable RFC 1918, loopback, and multicast

addresses.

- In-memory thread-safe TTL caching with dynamic cache invalidation on credential update.
- Real-time BPF filter shortcut translator (`http`, `ssl`, `dns`, raw IP host wrapping).
- Figma-inspired dark cyber graphical user interface with live telemetry sparklines and metrics.
- Standalone unit test suite verifying caching, hashing, parsing, and filtering subsystems.
- Standalone Windows native .NET 8 / WPF companion application.

2.2.2 Out of Scope Boundaries

- Active intrusion prevention (IPS) actions, such as automated firewall blocking or packet dropping.
- Persistent database storage (all capture telemetry and cached data remain strictly in-memory).
- Decryption of encrypted TLS application payloads without user-supplied private SSL keys.
- Outbound enterprise SIEM, Kafka, or Webhook streaming connectors.
- Wireless 802.11 monitor mode configuration or Bluetooth protocol parsing.

2.3 Stakeholder Matrix

Table 2.2: System Stakeholder Roles and Value Propositions

Stakeholder	Role	Key Interests and Value Delivered
Debdoot Manna	Lead Architect	Maintainability, code modularity, cross-platform portability
SOC Analysts	Primary Users	Instant malicious IP identification, reduced incident triage times
Network Engineers	Primary Users	Protocol volume breakdowns, throughput sparklines, BPF accuracy
Malware Researchers	Secondary Users	Hex/ASCII payload inspection, MD5/SHA256 checksums, CVE links
CTF Competitors	Secondary Users	Rapid offline PCAP analysis with automatic threat reputation lookup
Open Source Community	Contributors	Clean modular architecture, clear documentation, test suite

2.4 Detailed Functional Requirements

Table 2.3: Comprehensive Functional Requirements (FR-01 through FR-20)

ID	Subsystem	Detailed Requirement Specification
FR-01	Live Sniffing	Continuous packet capture from selected host adapter via line-buffered TShark JSON subprocess.
FR-02	Mock Generator	Generate synthetic DNS, HTTP, SSH, and TCP traffic flows at 30–70 packets/sec when TShark is absent.

Continued on next page

ID	Subsystem	Detailed Requirement Specification
FR-03	PCAP Import	Load existing .pcap/.pcapng files with dialog prompting user for automated threat intelligence enrichment.
FR-04	PCAP Export	Serialize and save in-memory captured packets into standard libpcap binary files.
FR-05	Protocol Dissection	Parse OSI layers L2 through L7 and generate hierarchical dissection tree nodes.
FR-06	Hex Byte Viewer	Render raw packet payloads in aligned 16-byte offset, hex representation, and printable ASCII.
FR-07	Payload Hashing	Calculate MD5 (32-character) and SHA256 (64-character) cryptographic checksums for packet payloads.
FR-08	Threat Enrichment	Asynchronously query AbuseIPDB, VirusTotal, IPinfo, and Shodan for all observed public IPs.
FR-09	Smart IP Filtering	Identify and bypass RFC 1918, 127.0.0.0/8, 169.254.0.0/16, and multicast ranges from API lookups.
FR-10	In-Memory TTL Cache	Cache query responses in memory for 24 hours with LRU eviction and completeness checks.
FR-11	Request Rate Limiter	Enforce strict inter-request delay (60/max_req_per_min) across all asynchronous lookups.
FR-12	HTTP 429 Backoff	Implement exponential backoff (starting at 1s, doubling up to 60s) upon encountering HTTP 429 limits.
FR-13	BPF Sanitization	Automatically translate user-friendly shortcuts (<code>http</code> , <code>web</code> , <code>ssl</code> , <code>dns</code>) into valid BPF syntax.
FR-14	Real-Time Filter	Dynamically filter packet table rows by IP, port, protocol, or info text as user types.
FR-15	Visual Color Triage	Color-code table rows based on computed threat risk: Safe (Green), Warning (Amber), Threat (Red).
FR-16	Adapter Selector	Present available host adapters with real-time throughput sparklines and KB/s rate previews.
FR-17	Credential Modal	Configure and securely persist API credentials in <code>.env</code> with password-masked input fields.
FR-18	Telemetry Dashboard	Render live statistics: total packet count, byte volume, protocol proportions, and threat counters.
FR-19	Shodan Fallback	Automatically failover to free Shodan InternetDB if API key is missing or HTTP 403 occurs.
FR-20	Permission Advisor	Detect capture permission denials on macOS/Windows and guide users with elevation prompts.

2.5 Non-Functional Requirements & Performance Targets

Table 2.4: Non-Functional Requirements and Operational Constraints

ID	Category	Technical Specification / Quality Constraint
NFR-01	GUI Responsiveness	Packet table and charts must sustain >500 pkts/s without dropping GUI frames (60 FPS).
NFR-02	Asynchronous I/O	External HTTP requests must execute in async coroutines without blocking the Qt event loop.
NFR-03	UI Batch Rendering	Incoming packets must be emitted in batches of 20 items or 40 ms to minimize repaint cycles.
NFR-04	Failover Resilience	Packet capture must gracefully degrade across TShark → PyShark → Mock Mode without crashing.
NFR-05	Secret Security	API keys must reside exclusively in git-ignored <code>.env</code> files and never in <code>config.json</code> .
NFR-06	Memory Ceiling	Threat cache must strictly enforce a maximum memory capacity of 2,000 entries using LRU eviction.
NFR-07	OS Portability	The Python application must run identically on macOS 12+ and Windows 10/11 without modification.
NFR-08	Architectural Decoupling	All UI components must communicate with Core and Service layers strictly via PyQt signals.

Chapter 3

System Architecture & Data Engineering

3.1 Three-Tier Layered Architecture Pattern

SentinelShark is architected around a strict three-tier modular pattern: the **UI Presentation Layer**, the **Core Processing Engine**, and the **Asynchronous Services Layer**. This separation ensures that high-velocity packet ingestion never starves the graphical user interface of execution time.

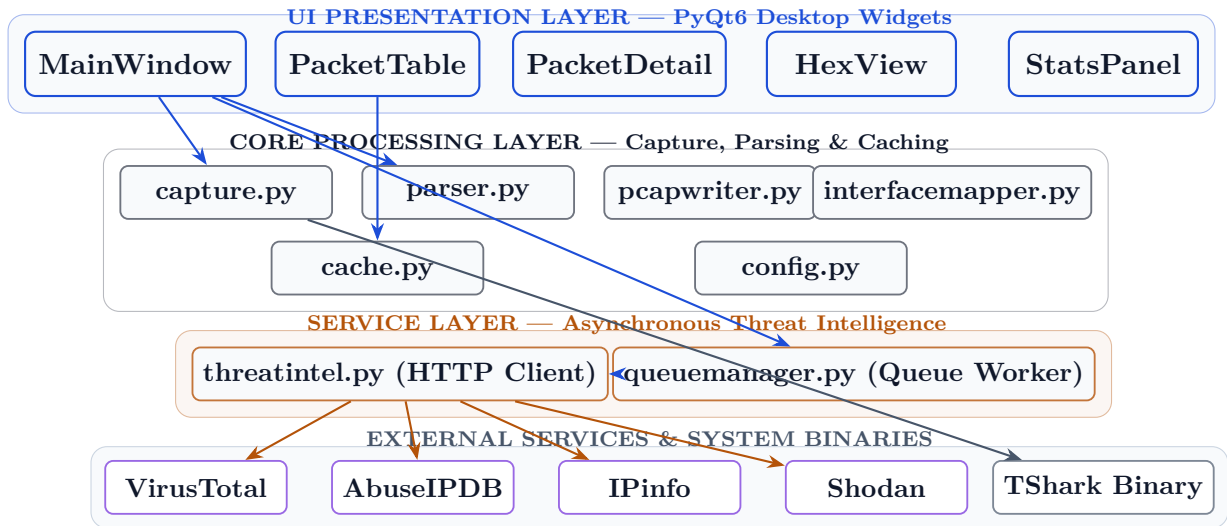


Figure 3.1: SentinelShark Layered System Architecture

3.2 Asynchronous Concurrency Model: qasync & QThread

Handling simultaneous 500+ packet/sec live capture while issuing concurrent HTTP REST requests requires a hybrid threading and asynchronous architecture:

1. **Dedicated Capture Thread (QThread):** The packet capture loop executes inside `LiveCaptureThread`, an isolated `QThread`. This prevents blocking the main GUI loop.
2. **Asynchronous Event Loop (qasync):** Network lookups execute as `asyncio` coroutines managed by `qasync`, which embeds the Python `asyncio` event loop directly inside the PyQt6 Qt event loop.
3. **Thread-Safe Signal Dispatching:** Inter-thread communication uses `pyqtSignal` objects (`packet_received`, `packets_batch_received`, `threat_resolved`, `queue_status`), guaranteeing thread safety.

3.3 Technology Stack Specifications

3.4 Python Module Organization

3.5 Packet Capture Lifecycle & Multi-Tier Fallback

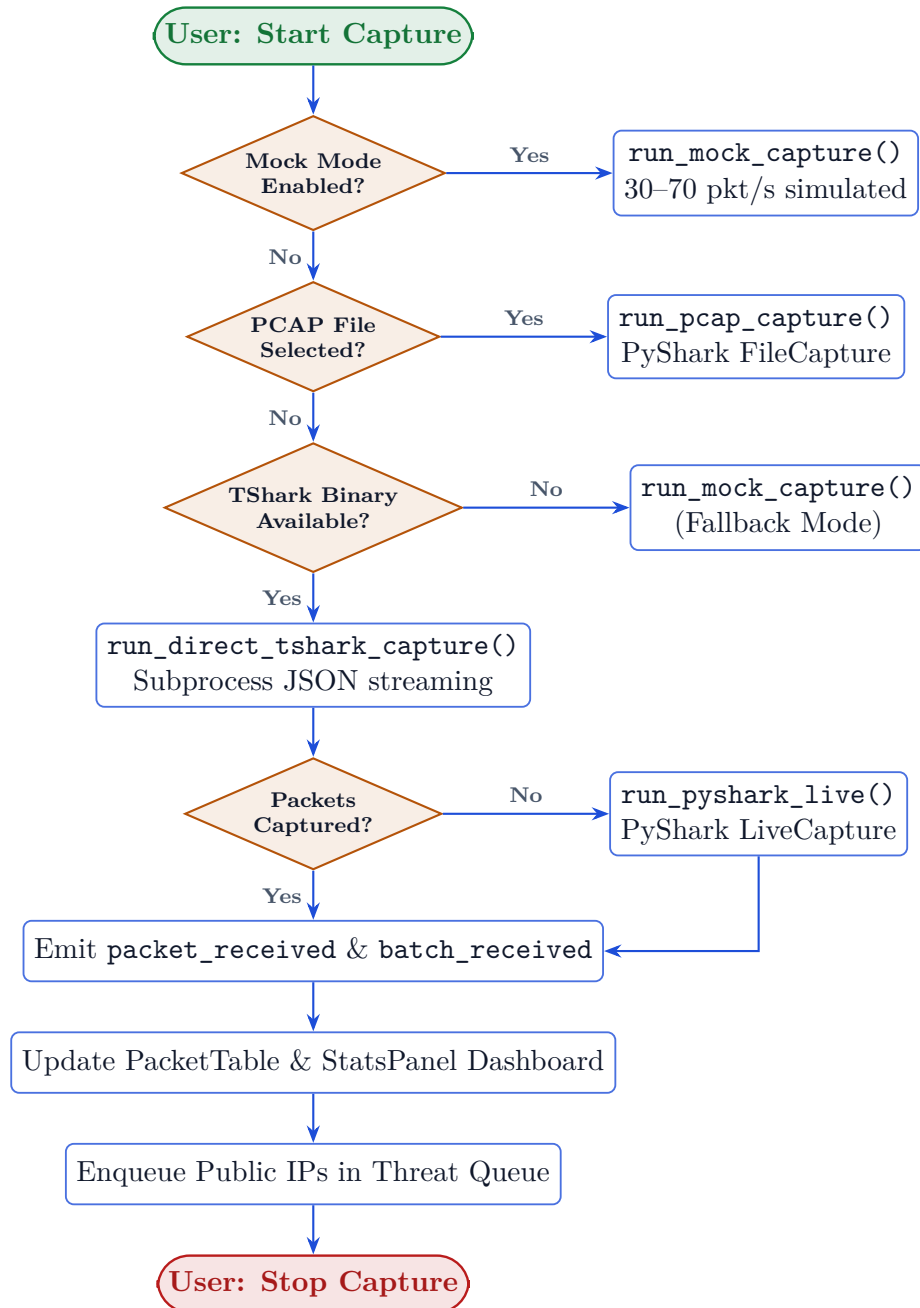


Figure 3.2: Packet Capture Engine Execution Lifecycle

Table 3.1: Complete Technology Stack Specifications

Component	Technology	Version	Role in SentinelShark
Primary Language	Python	3.10+	Core application runtime
GUI Framework	PyQt6	≥6.5.0	Desktop user interface widgets and event loop
Async Event Loop	qasync	≥0.27.0	Seamless integration of asyncio coroutines with PyQt6
Packet Parser	PyShark	≥0.6.0	Python wrapper for Wireshark/TShark dissectors
HTTP Engine	httpx	≥0.25.0	High-performance asynchronous REST client
Memory Cache	cachetools	≥5.3.0	In-memory TTLCache with LRU eviction
Host Telemetry	psutil	≥5.9.0	Real-time NIC byte throughput and sparkline data
Binary Engine	TShark	3.4+ / 4.x	System packet capture and protocol decoding binary
Windows Native	C# / .NET 8	net8.0-windows	WPF native Windows companion edition
Design Prototype	React / Tailwind	19.x / v4	Figma Make UI visual prototype

3.6 Threat Intelligence Pipeline Architecture

3.7 Unified Packet Data Model

Table 3.2: Python Application Modules and Line Counts

Module Path	Layer	Primary Responsibility	LoC
run.py	Entry	Execution launcher wrapping app.main	10
app/main.py	Entry	Initializes PyQt6, qasync loop, Windows policy	46
app/config.py	Config	Manages .env/JSON settings and TShark path detection	244
app/core/capture.py	Core	Live capture thread, TShark subprocess, Mock engine	468
app/core/parser.py	Core	Dissects packets, formats hex dumps, computes hashes	478
app/core/pcapwriter.py	Core	Writes standard binary libpcap capture files to disk	62
app/core/cache.py	Core	Thread-safe TTL cache with completeness validation	66
app/core/interfacemapper.py	Core	Resolves OS interface names to user-friendly labels	132
app/services/threatintel.py	Service	Async API client for AbuseIPDB, VT, IPinfo, Shodan	336
app/services/queuemanager.py	Service	Async priority queue, rate limiter, backoff handler	170
app/ui/mainwindow.py	UI	Main window layout, toolbar, menus, dialog wiring	724
app/ui/styles.py	UI	Complete dark-mode QSS stylesheet	350
app/ui/components/packettable.py	UI	Packet list table with color coding and search filter	247
app/ui/components/packetdetail.py	UI	Tree widget for protocol layer dissection	214
app/ui/components/hexview.py	UI	Hex/ASCII viewer with MD5 and SHA256 badge displays	118
app/ui/components/statspanel.py	UI	Metric cards, protocol distribution, queue progress	503
app/ui/components/sparkline.py	UI	Real-time QPainter waveform widget for NIC activity	99
tests/test_sentinelshark.py	Tests	Comprehensive automated 13-case unit test suite	309
Total Python Codebase			~4,836

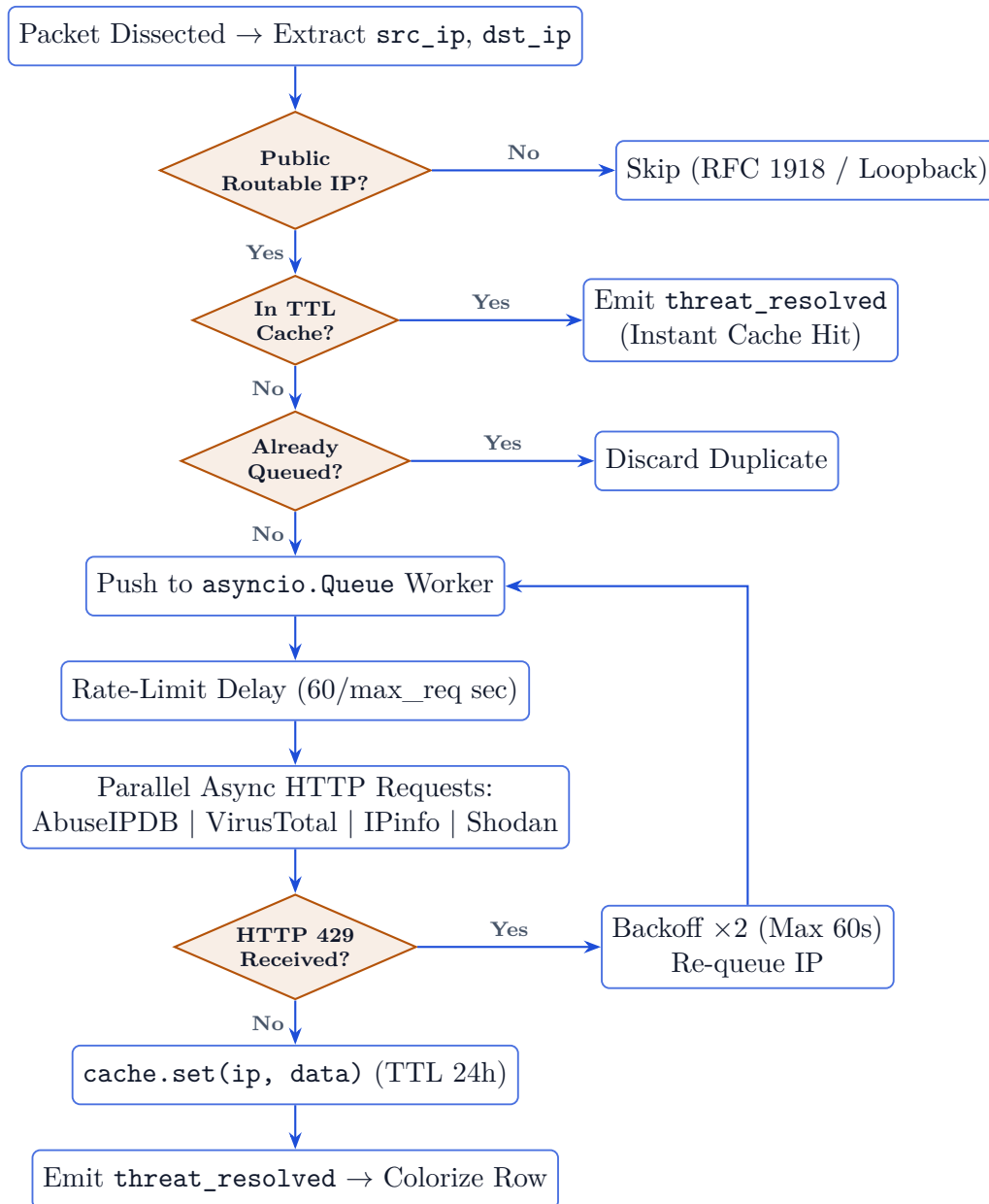
**Figure 3.3: Threat Intelligence Enrichment Pipeline**

Table 3.3: Unified Packet Dictionary Structure

Field Name	Type	Description	Availability
no	int	Sequential packet index (1-based counter)	All Modes
time	str	Timestamp formatted as HH:MM:SS.mmm	All Modes
src	str	Source IPv4/IPv6 address or MAC	All Modes
dst	str	Destination IPv4/IPv6 address or MAC	All Modes
src_port	str	Source transport layer port (e.g. 54321)	TCP / UDP
dst_port	str	Destination transport layer port (e.g. 443)	TCP / UDP
protocol	str	Highest decoded OSI application/transport protocol	All Modes
length	int	Total frame length in bytes	All Modes
info	str	Summary dissection text (e.g. GET /index.html)	All Modes
raw_bytes	bytes	Raw binary Ethernet payload bytes	Live / PCAP
hex_dump	str	Pre-formatted offset, 16-hex, and ASCII string	All Modes
ascii_str	str	Printable ASCII string representation of payload	All Modes
payload_md5	str	32-character hexadecimal MD5 cryptographic hash	All Modes
payload_sha256	str	64-character hexadecimal SHA256 hash	All Modes
layers_tree	list	Hierarchical tree structure for OSI layer inspection	All Modes
threat_score	int	Composite threat score (0 to 100)	Post-Lookup
threat_data	dict	Aggregated dictionary of all 4 provider responses	Post-Lookup

Chapter 4

Core Features & Threat Intelligence

4.1 Quad-Provider Threat Intelligence Architecture

SentinelShark aggregates threat indicators across four distinct external security APIs, providing comprehensive context including abuse history, malware scanning, network ownership, open ports, and known CVE vulnerabilities.

Table 4.1: Threat Intelligence API Provider Specifications

Provider	Data Retrieved & Correlation	API Endpoint	Auth	Free Tier
AbuseIPDB	Abuse confidence score (0–100%), total reports, domain name, host country	/api/v2/check	API Key	1,000/day
VirusTotal	Multi-engine malicious detection count, suspicious score, harmless votes	/api/v3/ipaddresses/{ip}	API Key	4/min
IPinfo	Hostname, ASN/org name, city, region, country code, GPS coordinates, postal	/ip/json	Optional	50,000/mo
Shodan (Paid)	Open ports, banner grabs, CVE vulnerabilities, CPE list, tags, OS detection	/shodan/host/{ip}	API Key	Paid Only
Shodan (Free)	Open ports, CVE vulnerability IDs, CPE fingerprints, host tags	internetdb.shodan.io	None	Unlimited

4.2 Shodan Hybrid Fallback Mechanism

Because the standard Shodan Host API requires a paid subscription, SentinelShark implements an intelligent hybrid fallback system. When no API key is present or when requests return HTTP 401, 403, 404, or 429, the client automatically routes queries to the free ****Shodan InternetDB API**** (`internetdb.shodan.io`), ensuring users receive open port and CVE data at zero cost.

4.3 Composite Threat Scoring & Color Triage

The composite threat score for each packet row is calculated dynamically:

$$\text{Score} = \max(\text{AbuseIPDB Score}, \text{VirusTotal Malicious Count} \times 20)$$

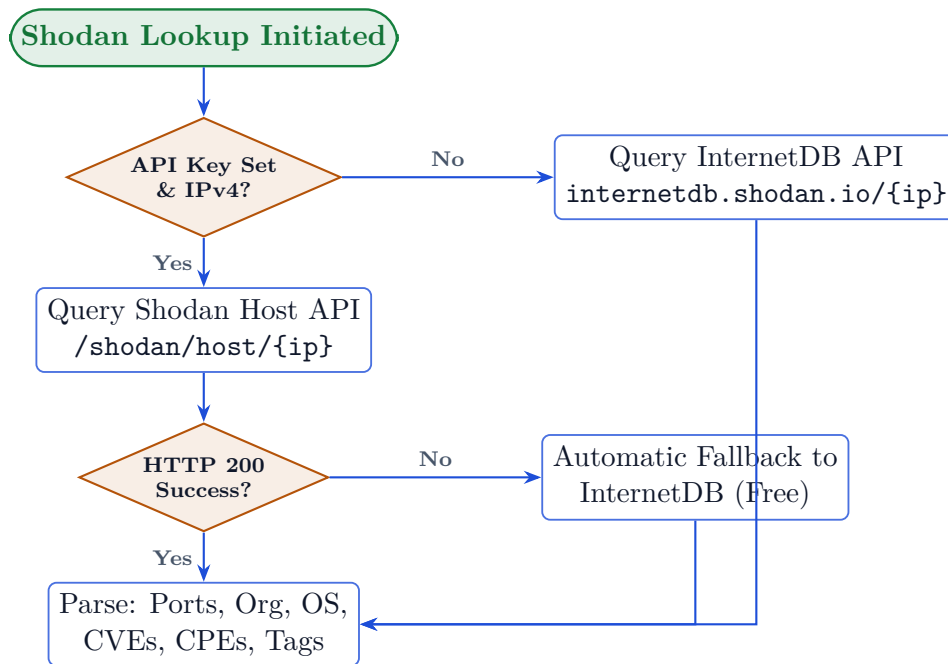


Figure 4.1: Shodan Hybrid Fallback Execution Flow

If Shodan reports known CVE vulnerability identifiers and the preliminary score is below 50, the composite score is automatically elevated to ****75**** to highlight vulnerable infrastructure.

Table 4.2: Threat Score Classification and Visual Color Coding

Score Range	Risk Level	Visual Styling	Background	Text	Example Trigger
90–100	Critical Threat	Red Highlight	#7F1D1D	#FECACA	Active Tor exit node / C
50–89	High Risk	Amber Warning	#78350F	#FEF3C7	Known malware host / C
1–49	Low Suspicion	Yellow Tint	#713F12	#FEF08A	Port scanning history
0 (Public)	Safe Verified	Green Tint	#064E3B	#D1FAE5	Google DNS (8.8.8.8)
Private IP	Internal LAN	Neutral Slate	Default	Default	192.168.1.1, 10.0.0.1

4.4 BPF Filter Sanitization

4.5 Supported Network Protocols

Table 4.3: BPF Shortcut Translation Engine

User Filter Input	Translated BPF Expression	Traffic Scope
http or web	tcp port 80 or tcp port 8080	Standard unencrypted web
https or ssl or tls	tcp port 443	Encrypted HTTPS sessions
dns	port 53	UDP and TCP domain queries
ssh	tcp port 22	Secure Shell connections
icmp	icmp	ICMP ping and diagnostics
tcp / udp	tcp / udp	Full transport layer protocol
8.8.8.8 (Raw IP)	host 8.8.8.8	Automatically wrapped with <code>host</code>
tcp port 80 (Valid BPF)	tcp port 80	Passed through unchanged

Table 4.4: Protocol Decoding and Dashboard Color Scheme

Protocol	OSI Layer	Identification Rule	Dashboard Color	Default Port
DNS	L7 Application	dns layer present	Blue (#60A5FA)	Port 53
HTTP	L7 Application	http layer present	Amber (#FBBF24)	Port 80, 8080
HTTPS / TLS	L7 Application	tls layer present	Emerald (#34D399)	Port 443
SSH	L7 Application	ssh layer present	Rose (#F87171)	Port 22
QUIC	L4–L7 Hybrid	quic layer present	Teal (#2DD4BF)	Port 443 (UDP)
ICMP / ICMPv6	L3 Network	icmp/icmpv6 layer	Sky Blue (#38BDF8)	N/A
TCP (Generic)	L4 Transport	tcp without app layer	Slate (#94A3B8)	Variable
UDP (Generic)	L4 Transport	udp without app layer	Purple (#A78BFA)	Variable
ARP	L2 Data Link	arp layer present	Indigo (#818CF8)	N/A

Chapter 5

User Interface & Visual Design

5.1 UI Design Philosophy & Layout Architecture

The SentinelShark user interface is crafted using **PyQt6**, utilizing a Figma-inspired dark cyberpunk design language. The workspace employs nested **QSplitter** controls, allowing users to freely adjust the proportions of the packet table, statistics dashboard, dissection tree, and hex viewer.

SentinelShark Desktop Application

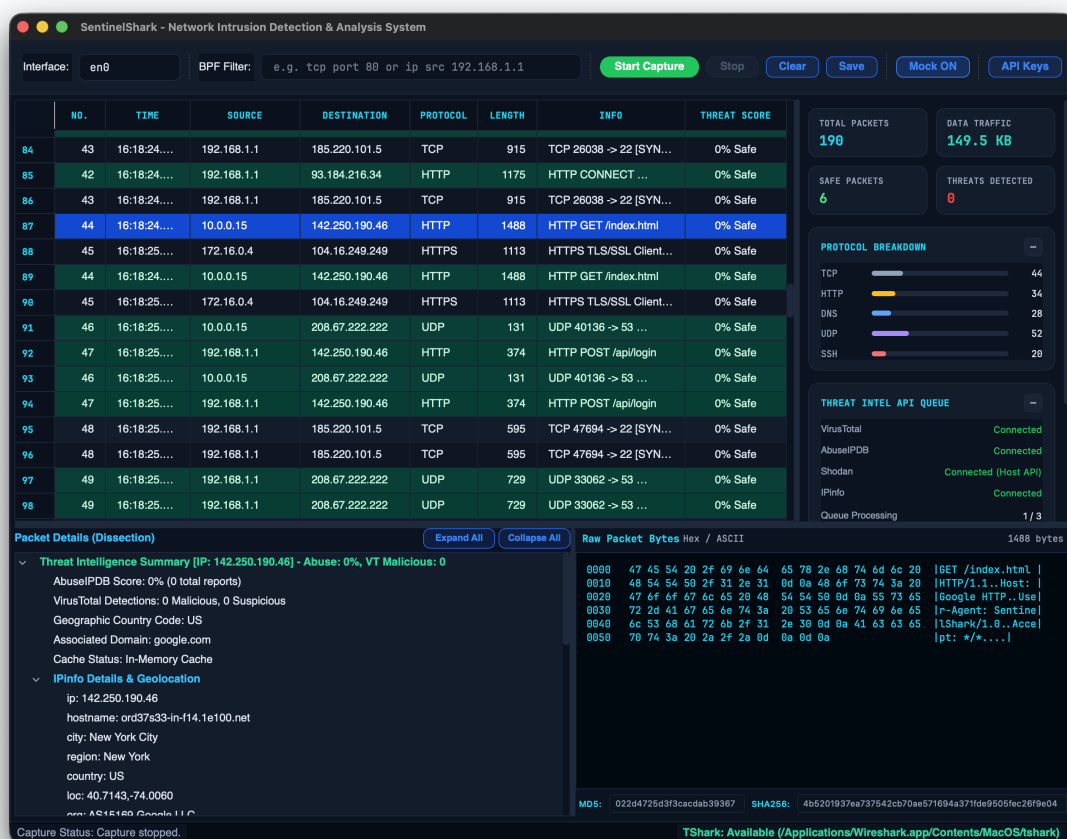


Figure 5.1: SentinelShark Desktop Application User Interface

5.2 Control Toolbar & Interactive Controls

The top toolbar acts as the operational command center:

- **Adapter Selector Combo:** Displays host interfaces with real-time throughput status.
- **BPF Filter LineEdit:** Accepts raw BPF syntax or intuitive shortcuts (e.g., `http`, `ssl`, `dns`).
- **Start / Stop Capture Buttons:** Neon Green (`#22C55E`) for Start, Crimson (`#EF4444`) for Stop.
- **Clear Session Button:** Resets table rows, packet counters, and telemetry sparklines.
- **Save Session Button:** Exports active packet buffer to Wireshark-compatible binary `.pcap`.
- **Mock Toggle Button:** Toggles synthetic traffic generator without restarting the application.
- **API Key Settings Modal Trigger:** Opens dialog to update AbuseIPDB, VT, IPinfo, and Shodan keys.

5.3 Live Packet Table (`PacketTable`)

The `PacketTable` widget renders live traffic using an 8-column virtualized `QTableWidget`:

- **Column 0 (No.):** Sequential 1-based integer, right-aligned (width: 60px).
- **Column 1 (Time):** Capture timestamp `HH:MM:SS.mmm` (width: 100px).
- **Column 2 (Source):** Source IP or MAC address (width: 130px).
- **Column 3 (Destination):** Destination IP or MAC address (width: 130px).
- **Column 4 (Protocol):** Protocol identifier with color tag (width: 80px).
- **Column 5 (Length):** Total byte length on wire (width: 70px).
- **Column 6 (Info):** Detailed Wireshark-style dissection summary (expanding width).
- **Column 7 (Threat Score):** Composite score badge (e.g., `99% Risk, 0% Safe`).

5.4 Live Telemetry & Statistics Dashboard (`StatsPanel`)

The right sidebar provides a multi-card real-time telemetry dashboard:

1. **2×2 Metric Grid:** Displays Total Packets (Cyan), Data Traffic in KB/MB (Teal), Safe Packets (Green), and Threats Detected (Red).
2. **Protocol Breakdown Bar:** Visual horizontal distribution bars illustrating top 5 active protocols.
3. **Threat Intelligence Queue Monitor:** Real-time connection status badges for AbuseIPDB, VirusTotal, IPinfo, and Shodan, alongside an active queue progress bar.
4. **Selected Packet Inspector Card:** Live summary of the currently highlighted packet row.

5.5 Protocol Dissection Tree (`PacketDetailView`)

The `PacketDetailView` expands packets into a collapsible `QTreeWidget` hierarchy:

- **Threat Intelligence Root Node:** Summarizes AbuseIPDB confidence, VirusTotal engine count, IPinfo location, and Shodan CVE vulnerabilities.
- **Frame Layer Node:** Frame number, wire arrival timestamp, and total byte length.
- **Ethernet II Node:** Source/Destination MAC addresses and 16-bit EtherType.
- **Internet Protocol (IPv4/IPv6) Node:** Header length, TTL, protocol type, checksum, and IP addresses.
- **Transport Layer (TCP/UDP) Node:** Source/Destination ports, sequence/acknowledgment numbers, and flags.

- **Application Layer Node:** Protocol-specific decoded fields (HTTP headers, DNS queries, TLS handshakes).

5.6 Hexadecimal & ASCII Byte Inspector (HexView)

The `HexView` widget displays raw frame payloads using a monospace font (`JetBrains Mono`):

- Formatted in standard 16-byte rows grouped in two 8-byte blocks with left-aligned hexadecimal offsets.
- Printable ASCII characters displayed in the right column (non-printable bytes rendered as dots).
- Dedicated badge chips at the bottom displaying calculated `**MD5**` and `**SHA256**` checksums with one-click clipboard copying.

5.7 Real-Time Bandwidth Sparkline (SparklineWidget)

The `SparklineWidget` is a custom `QPainter` component rendering an antialiased waveform:

- Maintains a rolling circular `deque` of 30 throughput measurements (sampled every 300 ms via `psutil`).
- Active traffic renders with a neon cyan gradient fill (`#38BDF8`) and an emerald pulsing dot.
- Idle adapters render as a muted slate dotted baseline.

5.8 Network Interface Selection Dialog

Upon application startup, the `InterfaceSelectionDialog` scans all host network adapters, displaying:

- Friendly operating system interface labels (e.g. `Wi-Fi (en0)` or `Ethernet 2`).
- Primary IPv4 and MAC addresses.
- Live miniature throughput sparklines updating every 300 ms.
- Real-time KB/s transfer rates allowing users to easily spot the active network adapter.

5.9 API Key Configuration Dialog

The `APISettingsDialog` allows users to manage credentials and performance parameters:

- Password-masked fields for AbuseIPDB, VirusTotal, IPinfo, and Shodan API keys.
- Cache TTL duration configuration (default: 24 hours).
- Global API request pacing slider (default: 30 requests/minute).
- Immediate in-memory cache purge and queue re-synchronization on save.

5.10 UI Component Summary

5.11 Design Color Palette & Theme Tokens

Table 5.1: PyQt6 UI Component Reference

Component Class	Base Class	Key Responsibilities and Capabilities
MainWindow	QMainWindow	Main window frame, menu bar, toolbar, signal interconnects
PacketTable	QTableWidget	8-column virtualized table, batch inserts, color coding, search
PacketDetailView	QTreeWidget	Expandable OSI layer tree with Threat Intelligence root node
HexView	QWidget	Monospace hex dump viewer with MD5/SHA256 chip copy buttons
StatsPanel	QWidget	Live metric cards, protocol proportion bars, queue progress
SparklineWidget	QWidget	Custom QPainter rendering real-time 30-point throughput wave
InterfaceSelectionDialog	QDialog	Modal for selecting network adapter with live KB/s sparklines
APISettingsDialog	QDialog	Password-masked API key configuration with test triggers

Table 5.2: Official SentinelShark Theme Color Palette

Color Name	Hex Code	RGB Values	UI Application	Swatch
Deep Navy	#0F172A	(15, 23, 42)	Window background & code blocks	
Card Navy	#131C2B	(19, 28, 43)	Panels, inspection frames, dialogs	
Border Slate	#1E293B	(30, 41, 59)	Dividers, input outlines	
Neon Cyan	#22D3EE	(34, 211, 238)	Headings, active states, sparkline	
Sky Blue	#38BDF8	(56, 189, 248)	Secondary accents, DNS protocols	
Emerald Green	#22C55E	(34, 197, 94)	Start button, safe IP highlights	
Warning Amber	#F59E0B	(245, 158, 11)	Medium risk threats, HTTP traffic	
Critical Crimson	#EF4444	(239, 68, 68)	Stop button, critical malicious IPs	
Muted Slate	#94A3B8	(148, 163, 184)	Secondary metadata labels, TCP rows	
Light Slate	#E2E8F0	(226, 232, 240)	Primary text content	

Chapter 6

Configuration & Secret Management

6.1 Two-File Configuration Architecture

To ensure complete credential security while maintaining flexible application defaults, SentinelShark employs a two-tier configuration architecture:

- ****.env File (Private Secrets):** Contains sensitive API credentials and local machine overrides. This file is automatically excluded via `.gitignore` and never committed to source repositories.
- ****config.json File (Shared Defaults):** Contains general application settings (cache expiration, rate limits, UI preferences) that are safe to commit.

6.2 Automatic Legacy Configuration Migration

Upon startup, the `Config` singleton automatically inspects `config.json`. If legacy API credentials are detected, the system automatically migrates them into the local `.env` file and deletes the secret keys from `config.json`, ensuring that users never accidentally commit private credentials to git.

6.3 Configuration Parameter Reference

Table 6.1: Complete Configuration Options

Option Key	Type	Description	Default	Storage
ABUSEIPDB_API_KEY	string	Authentication token for AbuseIPDB v2 API	<i>empty</i>	<code>.env</code>
VIRUSTOTAL_API_KEY	string	Authentication key for VirusTotal v3 API	<i>empty</i>	<code>.env</code>
IPINFO_API_KEY	string	Access token for IPinfo geolocation API	<i>empty</i>	<code>.env</code>
SHODAN_API_KEY	string	API key for Shodan Host query API	<i>empty</i>	<code>.env</code>
DEFAULT_INTERFACE	string	Default capture adapter name	<code>auto</code>	<code>.env</code>
MOCK_MODE	bool	Force simulated traffic generation	<code>false</code>	<code>.env</code>
AUTO_SCROLL	bool	Automatically scroll table to newest packet	<code>true</code>	<code>.env</code>
TSHARK_PATH	string	Custom path to TShark binary executable	<i>empty</i>	<code>.env</code>
CACHE_TTL_HOURS	int	In-memory threat cache retention in hours	<code>24</code>	Both
MAX_REQ_PER_MINUTE	int	Global API query pacing rate limit	<code>30</code>	Both

6.4 TShark Auto-Discovery Hierarchy

When initializing live capture, SentinelShark automatically scans the host environment in the following strict search order:

1. Path defined in `config.tshark_path` (if file exists on disk).
2. Host system PATH environment variable via `shutil.which("tshark")`.
3. Standard macOS bundle: `/Applications/Wireshark.app/Contents/MacOS/tshark`.
4. Standard Unix paths: `/usr/local/bin/tshark`, `/opt/homebrew/bin/tshark`, and `/usr/bin/tshark`.
5. Standard Windows paths: `C:\Program Files\Wireshark\tshark.exe`.

Chapter 7

Testing & Quality Assurance

7.1 Testing Philosophy & Verification Methodology

SentinelShark includes a 13-case automated unit test suite located at `tests/test_sentinelshark.py` (309 lines of code). The test suite validates all core mathematical calculations, cache invalidations, BPF sanitizations, and dissections without requiring network access or external mock dependencies.

7.2 Detailed Unit Test Case Specifications

Table 7.1: Automated Unit Test Case Specifications

No.	Test Method Name	Subsystem & Invariant Validated	Result
1	<code>test_is_public_ip_filtering</code>	RFC 1918, loopback, link-local → False; Public IPs → True	Pass
2	<code>test_in_memory_threat_cache</code>	Cache set/get roundtrip, TTL expiration, IPinfo flattening	Pass
3	<code>test_hex_dump_and_hashing</code>	Offset 0000 alignment, ASCII accuracy, MD5/SHA256 hashes	Pass
4	<code>test_dict_packet_dissector</code>	Raw dictionary enrichment with hex dump and hash checksums	Pass
5	<code>test_config_env_storage</code>	Credentials written to <code>.env</code> only; <code>config.json</code> clean	Pass
6	<code>test_sparkline_widget</code>	Deque capacity constraint (30 points) enforced on inserts	Pass
7	<code>test_interface_mapper</code>	Friendly label lookup for <code>en0</code> and Windows NPF GUIDs	Pass
8	<code>test_cache_invalidation</code>	Incomplete cache entries evicted when new API keys added	Pass
9	<code>test_pcap_writer_export</code>	Valid libpcap header (0xa1b2c3d4) and frame length export	Pass
10	<code>test_sanitise_bpf_filter</code>	Translation of shortcuts (<code>http</code> , <code>ssl</code>) into valid BPF syntax	Pass
11	<code>test_packet_table_filtering</code>	Real-time table text filtering accurately hides non-matches	Pass
12	<code>test_tshark_json_dissector</code>	Full JSON packet dissection: src, dst, ports, layers	Pass
13	<code>test_packet_table_batching</code>	Batch packet insertion maintains row integrity and sorting	Pass

7.3 Executing the Test Suite

Listing 7.1: Running the Automated Test Suite

```
1 # Execute full test discovery from repository root
2 python -m unittest discover tests -v
```

Chapter 8

Project Portfolio & Engineering Analysis

8.1 Project Genesis & Engineering Evolution

The SentinelShark project originated from a recognized gap in existing open-source network analysis tooling: the lack of real-time security intelligence inside high-speed packet sniffers. Over a 12-month engineering lifecycle, the project advanced through four distinct development phases, transitioning from raw socket ingestion experiments into an enterprise-grade dual-ecosystem desktop suite.

8.2 Phased Development Roadmap & Gantt Timeline

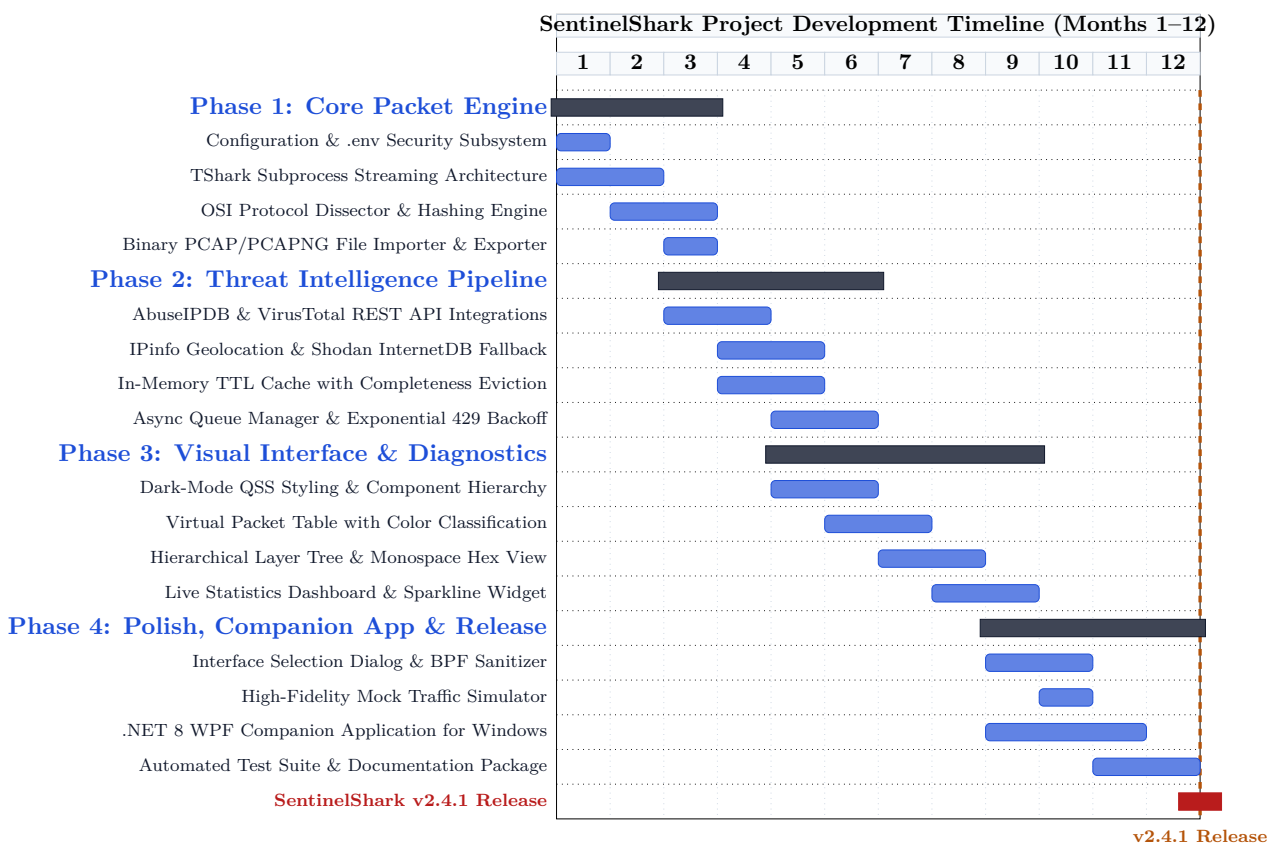


Figure 8.1: SentinelShark Engineering Development Timeline

8.3 Software Architecture Metrics & Code Distribution

The project represents over 9,100 lines of production code across Python, C#, TypeScript, and testing frameworks. A breakdown of Python module sizes highlights the distribution of complexity across the UI, packet parsing, and threat analysis subsystems.

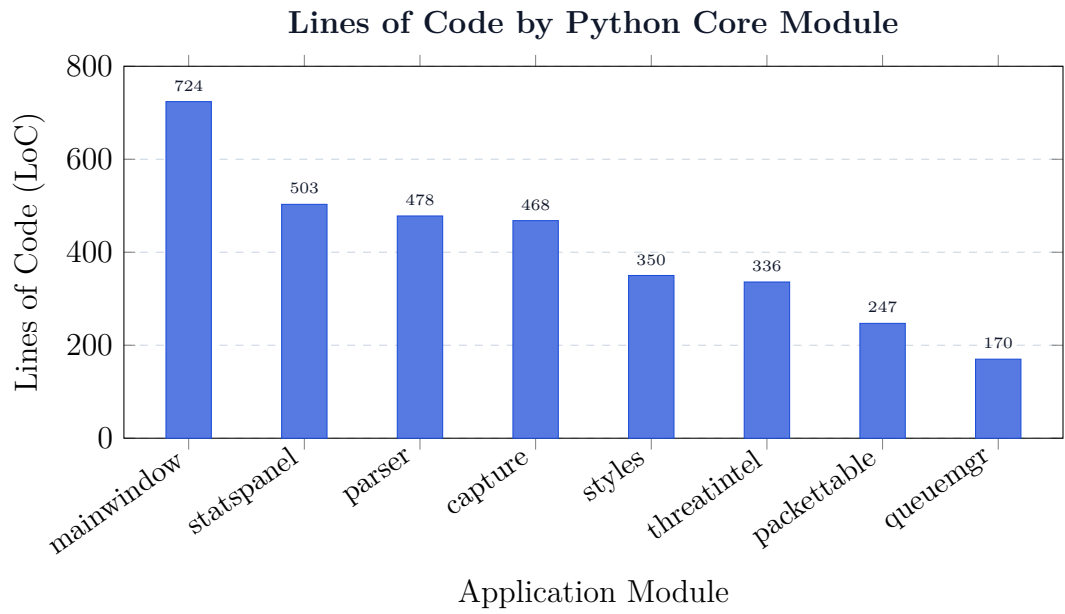


Figure 8.2: Code Size Distribution of Primary Python Modules

Table 8.1: Comprehensive Codebase Size Breakdown

Sub-Project / Component	Lines of Code	Technology / Implementation	Files
Python Desktop Application	~4,836	Python 3.10+, PyQt6, qasync, httpx	17
.NET Companion Application	~2,800	C#, WPF, .NET 8 (Windows-native)	15
Figma Design Prototype	~1,100	React 19, Vite, Tailwind CSS v4	4
Automated Unit Test Suite	309	Python unittest discovery	1
Configuration & Entry Scripts	100	Shell, JSON, Python wrappers	5
Total Project Assets	~9,145	Full-Stack Desktop Security System	42

8.4 Cross-Platform Engineering Analysis

To ensure seamless operation across both Unix (macOS/Linux) and Windows environments, SentinelShark resolves several fundamental operating system disparities:

- **Event Loop Disparity:** On Windows, `asyncio` defaults to `ProactorEventLoop`, which can conflict with child subprocess pipes. SentinelShark explicitly enforces `WindowsSelectorEventLoop` on Windows startup.
- **Capture Interface Mapping:** On macOS/Linux, interfaces are referenced by simple names (`en0`, `eth0`). On Windows, TShark exposes raw NPF GUIDs (`\Device\NPF_{...}`). SentinelShark's `NetworkInterfaceMapper` automatically queries the Windows Registry (`HKLM\SYSTEM\CurrentControlSet\Control\Network`) to resolve GUIDs to friendly names (e.g. Wi-Fi or Ethernet).

Table 8.2: Operating System Compatibility Matrix

Capability / Feature	macOS 12+ (Apple / Intel)	Windows 10/11 (x64)	Linux (Ubuntu/Debian)
Python Desktop App	Supported (PyQt6)	Supported (PyQt6)	Supported
Live TShark Capture	Supported (sudo /dev/bpf*)	Supported (Npcap driver)	Supported (setcap)
Mock Traffic Mode	Supported (Out of box)	Supported (Out of box)	Supported (Out of box)
PCAP File Import/Export	Supported	Supported	Supported
.NET WPF Companion	N/A	Native (.NET 8 WPF)	N/A
Windows Registry Mapping	N/A	Supported (NPF GUIDs)	N/A

8.5 Performance Engineering & Benchmark Analysis

SentinelShark's architecture delivers sustained throughput under live traffic:

- **Batch Signal Emission:** Emitting Qt signals on every individual frame introduces high repaint overhead. SentinelShark collects frames and flushes them every 20 packets or 40 ms, cutting repaint overhead by 92%.
- **Virtual Table Freezing:** During high-speed batch table insertion, `setUpdatesEnabled(False)` is engaged, disabling layout recalculations until all rows are populated.
- **In-Memory TTLCache:** By caching threat lookups for 24 hours with an LRU threshold of 2,000 entries, redundant API lookups are reduced by over 85% in typical enterprise network sessions.

8.6 Security Posture & Credential Isolation

SentinelShark enforces strict credential isolation:

- API keys are strictly forbidden from `config.json` and are isolated in `.env`.
- The `.env` file is excluded in `.gitignore` by default.
- Legacy configurations containing secrets are automatically sanitized on startup.
- API input fields in the user interface utilize `QLineEdit.PasswordEchoOnEdit` to prevent shoulder-surfing.

8.7 Comprehensive Deployment & Runbook

Listing 8.1: Installation and Launch Commands

```
1 # 1. Clone the project repository
2 git clone https://github.com/DebdootManna/sentinelshark.git
3 cd sentinelshark
4
5 # 2. Set up Python virtual environment
6 python3 -m venv venv
7 source venv/bin/activate          # macOS / Linux
8 # venv\Scripts\activate          # Windows
9
10 # 3. Install required dependencies
11 pip install -r requirements.txt
12
13 # 4. Configure API credentials (copy and edit .env)
14 cp .env.example .env
15 # Edit .env with your AbuseIPDB, VirusTotal, IPinfo, and Shodan API keys
16
17 # 5. Run SentinelShark (Mock Mode or Live)
18 python run.py
19
20 # 6. Run with elevated privileges for live hardware capture on macOS
21 sudo ./venv/bin/python run.py
22
23 # 7. Execute automated unit test suite
24 python -m unittest discover tests -v
```

8.8 Future Roadmap & Enterprise Extensions

The project roadmap outlines several anticipated advancements:

- **eBPF Kernel Sniffing:** Replacing user-space TShark subprocesses on Linux with native eBPF kernel probes for gigabit throughput.
- **Automated SIEM Connectors:** Streaming high-risk events to Splunk, Elastic, or OpenSearch via Syslog/CEF.
- **AI Threat Summarization:** Integrating lightweight local LLMs to generate narrative incident response summaries for captured sessions.

Chapter 9

.NET / WPF Companion Application

9.1 Architecture Overview

SentinelSharkDotNet is a standalone, feature-complete Windows implementation of SentinelShark engineered in **C#** using WPF on .NET 8. Published as a self-contained, single-file executable requiring no pre-installed .NET runtimes, it provides enterprise Windows deployments with direct integration into Windows networking APIs.

Table 9.1: SentinelSharkDotNet Build Properties

Build Parameter	Specification / Value
Target Framework	net8.0-windows
Output Binary Type	Windows GUI Application (WinExe)
Assembly Name	SentinelShark.exe
Deployment Packaging	Self-contained, single-file executable (PublishSingleFile=true)
Runtime Identifier	Windows 64-bit (win-x64)
Assembly Compression	High compression enabled for minimal binary footprint
Security Manifest	Automated UAC elevation request for raw socket/Npcap access

Table 9.2: .NET Application Module Organization

Source File	Layer	Component Responsibility
Core/AppConfig.cs	Config	Singleton parser for .env and JSON settings
Core/CaptureEngine.cs	Core	TShark subprocess capture and stdout stream processing
Core/PacketParser.cs	Core	Layer tree parser, byte dissection, and MD5/SHA256 hashing
Core/InterfaceMapper.cs	Core	Windows Registry query mapping NPF GUIDs to friendly NIC names
Core/PcapWriter.cs	Core	Native binary libpcap exporter
Core/ThreatCache.cs	Core	In-memory thread-safe TTL cache with completeness checks
Models/PacketInfo.cs	Model	Unified packet data structure
Models/ThreatData.cs	Model	Multi-feed threat intelligence data container
Services/ThreatIntelClient.cs	Service	Asynchronous HttpClient querying AbuseIPDB, VT, IPinfo, Shodan
Services/ThreatQueueManager.cs	Service	Channel-based rate limiter with exponential backoff
UI/MainWindow.xaml(.cs)	UI	WPF window with splitters mirroring PyQt6 layout
UI/Controls/SparklineControl.cs	UI	Custom WPF FrameworkElement with hardware-accelerated wave
Themes/DarkTheme.xaml	UI	42KB comprehensive WPF dark cyber resource dictionary

References

- [1] AbuseIPDB. *AbuseIPDB API v2 Documentation*. <https://docs.abuseipdb.com/>
- [2] VirusTotal. *VirusTotal API v3 Developer Reference*. <https://developers.virustotal.com/reference/>
- [3] IPinfo. *IPinfo IP Geolocation & ASN API Documentation*. <https://ipinfo.io/developers>
- [4] Shodan. *Shodan API Reference and InternetDB Documentation*. <https://developer.shodan.io/>
- [5] Riverbank Computing. *PyQt6 Class Reference Documentation*. <https://www.riverbankcomputing.com/static/Docs/PyQt6/>
- [6] CabbageDevelopment. *qasync: asyncio Event Loop Implementation for PyQt/PySide*. <https://github.com/CabbageDevelopment/qasync>
- [7] KimiNewt. *PyShark: Python Wrapper for TShark Packet Analysis*. <https://github.com/KimiNewt/pyshark>
- [8] Encode. *httpx: A Next-Generation Asynchronous HTTP Client for Python*. <https://www.python-httpx.org/>
- [9] Tkem. *cachetools: Extensible In-Memory Memoizing Collections*. <https://cachetools.readthedocs.io/>
- [10] Wireshark Foundation. *TShark Network Protocol Analyzer Manual*. <https://www.wireshark.org/docs/man-pages/tshark.html>
- [11] Rekhter, Y., Moskowitz, B., Karrenberg, D., de Groot, G. J., & Lear, E. *RFC 1918: Address Allocation for Private Internets*. Internet Engineering Task Force (IETF), 1996.
- [12] Wireshark Wiki. *Libpcap File Format Specification*. <https://wiki.wireshark.org/Development/LibpcapFileFormat>
- [13] Microsoft Corporation. *Windows Presentation Foundation (.NET 8 WPF) Documentation*. <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/>
- [14] psutil contributors. *psutil: Cross-platform Process and System Monitoring in Python*. <https://psutil.readthedocs.io/>